



Project 4 Report

Real-Time Traffic Monitoring and Analysis

Ahmed Mohamed Abdelsalam

Mohamed Haitham Ezzat Yamani

Khaled Taha Taha Beheiry

Maged Mohamed Mohamed Mousa

Table of Contents

Part 1: SQL Database and Data Collection Tasks.....	4
Overview.....	4
Deliverables.....	4
SQL Database Schema	4
Table Design Details	4
1. Traffic_Sensors Table	4
2. Traffic_Logs Table	6
3. Incidents Table	7
Tools Used	8
Explanation of the Data Processing Code.....	8
1. Reading the Excel File.....	8
2. Checking the Number of Incidents	9
3. Generating Traffic Incident Speed Based on Weather Conditions	9
4. Cleaning the Dataset.....	10
5. Exporting to CSV	10
ETL Process: Extract, Transform, Load	11
1. Extract.....	11
2. Transform.....	11
3. Load.....	12
Full ETL Pipeline	12
Part 2: Data Warehouse and Python Integration	13
Create a Dedicated SQL Pool	14
Create SQL Database Server	15
Server Authentication	15
Configure Performance.....	16
ETL Process: Data Extraction, Transformation, and Loading	17
1. Extract Data from the Local Database	17
2. Open Connection with the Data Warehouse	18
3. Load Data into the Warehouse	19
4. Completion	19
Part 3: Traffic Analysis and Azure Services	20
Exploratory Data Analysis (EDA)	20
Correlation Between Numeric Fields	21
Analysis Based on Environmental Factors.....	21
Traffic Logs Analysis	24

Sensor Data Analysis	25
Part 4: Congestion predictive model:	28
Data Preparation and Feature Engineering	28
Data Representation	28
Congestion Score Calculation	28
Machine Learning Model for Prediction	29
Model Development	29
Model Evaluation	29
Real-time Traffic Congestion Prediction	29
Input Interface	29
Prediction Function	30
Data Preparation and Feature Engineering	31
Data Representation	31
Congestion Score Calculation	31
Machine Learning Model for Prediction	32
Model Development	32
Model Evaluation	32
Real-time Traffic Congestion Prediction	33
Input Interface	33
Prediction Function	34
References	34

Table of Figures

Figure 1:Traffic sensor data	5
Figure 2:Traffic logs data	6
Figure 3:incidents data	8
Figure 4:Create a Dedicated SQL Pool window in Azure	14
Figure 5:Create SQL Database Server window in Azure	15
Figure 6:Server Authentication in Azure	16
Figure 7:Configure Performance in Azure	17
Figure 8 Number of Traffic Incidents by Year	20
Figure 9 Incidents by Lighting Condition	22
Figure 10 Incidents by Weather Condition	22
Figure 11 Incidents by Road Characteristics	23
Figure 12 Distribution of Average Traffic Speeds	24
Figure 13 Total Vehicle Count Over Time	25
Figure 14 Traffic Sensor Status Distribution	26
Figure 15 Sensor Types Across Cities	27

Part 1: SQL Database and Data Collection Tasks

Overview

The objective of this project is to create a SQL database schema for managing traffic data using Microsoft SQL Server and SQL Management Studio. The database will store details of traffic sensors, logs of traffic data collected by the sensors, and reports of traffic-related incidents. Additionally, SQL queries will be created to analyze the traffic data, providing insights such as daily traffic summaries, incident reports, and sensor statistics.

Deliverables

1. **SQL Database Schema:** A comprehensive design for managing traffic data with populated tables for traffic sensors, logs, and incidents.
 2. **SQL Queries:** A set of SQL queries to analyze the traffic data, including daily traffic summaries, incident reports, and sensor statistics.
-

SQL Database Schema

The database schema includes three key tables:

- **Traffic_Sensors**
- **Traffic_Logs**
- **Incidents**

Table Design Details

1. Traffic_Sensors Table

This table stores details about the traffic sensors installed on various roadways.

Column	Data Type	Constraints	Description
SensorID	VARCHAR(50)	PRIMARY KEY	Unique identifier for each traffic sensor.
RoadwayID	VARCHAR(50)	NOT NULL, UNIQUE	Unique identifier for the roadway where the sensor is installed.
City	VARCHAR(100)	NOT NULL	The city where the sensor is located.
SensorType	VARCHAR(50)	NOT NULL	The type of traffic sensor (e.g., speed sensor, vehicle counter).
Status	VARCHAR(20)	NOT NULL	Current status of the sensor (e.g., Active, Inactive).

Constraints:

- **Primary Key:** SensorID is the unique identifier for each sensor.
- **Unique Constraint:** RoadwayID ensures that there is only one sensor for each roadway.

Example Data:

100 %					
Results Messages					
	SensorID	RoadwayID	City	SensorType	Status
1	sensor_1	1ST AVE	BLOOMINGTON	Vehicle Count Sensor	Active
2	sensor_10	CLEAR	BLOOMINGTON	Road Surface Sensor	Active
3	sensor_100	BELL TRACE	BLOOMINGTON	Speed Sensor	Inactive
4	sensor_101	BELL TRACE CIR	BLOOMINGTON	Vehicle Count Sensor	Inactive
5	sensor_102	BELLE AVE	ELLETTSVILLE	Air Quality Sensor	Active
6	sensor_103	BERNICE	BLOOMINGTON	Vehicle Count Sensor	Active
7	sensor_104	BETHEL LN	BLOOMINGTON	Weather Sensor	Inactive
8	sensor_105	BIRCH RD	BLOOMINGTON	Speed Sensor	Inactive
9	sensor_106	BLOOMFIELD RD	BLOOMINGTON	Infrared Sensor	Active
10	sensor_107	BLOOMFIELD RD	BLOOMINGTON	Pavement Condition Sensor	Active
11	sensor_108	BLUE RIDGE DR	BLOOMINGTON	Vehicle Count Sensor	Inactive
12	sensor_109	BLUEBIRD LN	BLOOMINGTON	Traffic Light Sensor	Active
13	sensor_11	4TH	BLOOMINGTON	Weather Sensor	Active
14	sensor_110	BOLIN LN	BLOOMINGTON	Weather Sensor	Inactive
15	sensor_111	BOLTINGHOUSE	BLOOMINGTON	Vehicle Count Sensor	Active

Figure 1: Traffic sensor data

2. Traffic_Logs Table

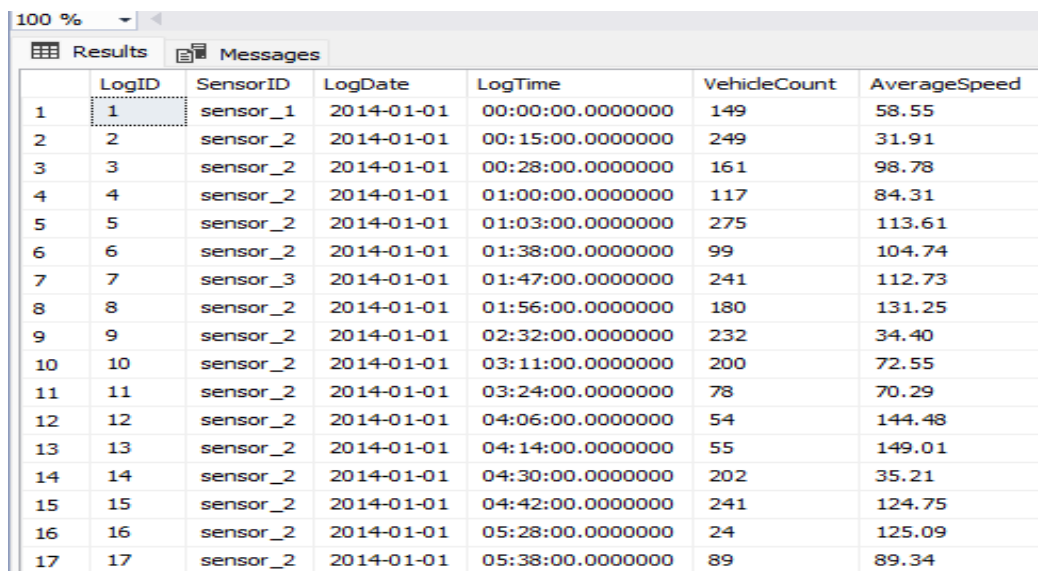
This table logs traffic data collected by the sensors at various times, storing details such as vehicle counts and average speed.

Column	Data Type	Constraints	Description
LogID	INT	PRIMARY KEY	Unique identifier for each traffic log entry.
SensorID	VARCHAR(50)	FOREIGN KEY	References the SensorID from the Traffic_Sensors table to associate logs with a specific sensor.
LogDate	DATE	NOT NULL	The date when the traffic data was recorded.
LogTime	TIME	NOT NULL	The time when the traffic data was recorded.
VehicleCount	INT	NULL	The number of vehicles counted by the sensor.
AverageSpeed	DECIMAL(5,2)	NULL	The average speed of the vehicles recorded by the sensor (in km/h).

Constraints:

- **Primary Key:** LogID uniquely identifies each log entry.
- **Foreign Key:** SensorID references the Traffic_Sensors table.

Example Data:



The screenshot shows a database query results window with a toolbar at the top containing 'Results' and 'Messages' tabs. The 'Results' tab is active, displaying a table with 7 columns: LogID, SensorID, LogDate, LogTime, VehicleCount, and AverageSpeed. The table contains 17 rows of data. The first row (LogID 1) is highlighted with a dashed border. The data shows logs from January 1, 2014, with various sensors and vehicle counts.

	LogID	SensorID	LogDate	LogTime	VehicleCount	AverageSpeed
1	1	sensor_1	2014-01-01	00:00:00.0000000	149	58.55
2	2	sensor_2	2014-01-01	00:15:00.0000000	249	31.91
3	3	sensor_2	2014-01-01	00:28:00.0000000	161	98.78
4	4	sensor_2	2014-01-01	01:00:00.0000000	117	84.31
5	5	sensor_2	2014-01-01	01:03:00.0000000	275	113.61
6	6	sensor_2	2014-01-01	01:38:00.0000000	99	104.74
7	7	sensor_3	2014-01-01	01:47:00.0000000	241	112.73
8	8	sensor_2	2014-01-01	01:56:00.0000000	180	131.25
9	9	sensor_2	2014-01-01	02:32:00.0000000	232	34.40
10	10	sensor_2	2014-01-01	03:11:00.0000000	200	72.55
11	11	sensor_2	2014-01-01	03:24:00.0000000	78	70.29
12	12	sensor_2	2014-01-01	04:06:00.0000000	54	144.48
13	13	sensor_2	2014-01-01	04:14:00.0000000	55	149.01
14	14	sensor_2	2014-01-01	04:30:00.0000000	202	35.21
15	15	sensor_2	2014-01-01	04:42:00.0000000	241	124.75
16	16	sensor_2	2014-01-01	05:28:00.0000000	24	125.09
17	17	sensor_2	2014-01-01	05:38:00.0000000	89	89.34

Figure 2:Traffic logs data

3. Incidents Table

This table logs details of traffic incidents, including involved vehicles, weather conditions, and contributing factors.

Column	Data Type	Constraints	Description
IncidentID	INT	PRIMARY KEY	Unique identifier for each incident report.
Agency	VARCHAR(50)	NOT NULL	The agency responsible for handling the incident.
City	VARCHAR(100)	NOT NULL	The city where the incident occurred.
IncidentDate	DATE	NOT NULL	The date of the incident.
IncidentTime	TIME	NOT NULL	The time of the incident.
VehiclesInvolved	INT	NULL	The number of vehicles involved in the incident.
Injuries	INT	NULL	The number of injuries resulting from the incident.
Fatalities	INT	NULL	The number of fatalities resulting from the incident.
RoadwayID	VARCHAR(50)	FOREIGN KEY	References the RoadwayID in the Traffic_Sensors table to associate incidents with roadways.
LightingCondition	VARCHAR(50)	NULL	Lighting conditions at the time of the incident (e.g., Daylight, Dark).
WeatherCondition	VARCHAR(50)	NULL	Weather conditions at the time of the incident (e.g., Rain, Clear).
SurfaceCondition	VARCHAR(50)	NULL	Surface condition of the roadway (e.g., Wet, Dry).
RoadCharacteristics	VARCHAR(50)	NULL	Characteristics of the road (e.g., Curved, Straight).
PrimaryFactor	VARCHAR(100)	NULL	Primary cause of the incident (e.g., Speeding, Drunk Driving).
TrafficIncidentSpeed	DECIMAL(5, 2)	NULL	The speed of traffic at the time of the incident (in km/h).

Constraints:

- **Primary Key:** IncidentID uniquely identifies each incident report.
- **Foreign Key:** RoadwayID references the Traffic_Sensors table to associate incidents with specific roadways.

Example Data:

IncidentID	Agency	City	IncidentDate	IncidentTime	VehiclesInvolved	Injuries	Fatalities	RoadwayID	LightingCondition	WeatherCondition	SurfaceCondition	RoadCharacteristics	PrimaryFactor	TrafficIncidentSpeed
1	MCSO	BLOOMINGTON	2014-07-30	01:16:00.0000000	1	0	0	1ST AVE	DARK	CLEAR	DRY	CURVE/GRADE	RAN OFF ROAD RIGHT	61.00
2	BPD	BLOOMINGTON	2015-05-11	09:50:00.0000000	2	0	0	1ST ST	DAYLIGHT	CLOUDY	DRY	STRAIGHT/LEVEL	OTHER (DRIVER)	56.00
3	BPD	BLOOMINGTON	2016-01-04	14:06:00.0000000	2	0	0	1ST ST	DAYLIGHT	CLEAR	DRY	NON-ROADWAY CRASH	UNSAFE BACKING	110.00
4	BPD	BLOOMINGTON	2016-06-14	15:00:00.0000000	2	0	0	1ST ST	DAYLIGHT	RAIN	WET	STRAIGHT/GRADE	IMPROPER TURNING	85.00
5	BPD	BLOOMINGTON	2013-08-06	00:00:00.0000000	1	1	0	1ST ST	DAYLIGHT	CLOUDY	DRY	STRAIGHT/LEVEL	DRIVER ILLNESS	89.00
6	BPD	BLOOMINGTON	2013-09-03	00:00:00.0000000	2	1	0	1ST ST	DAYLIGHT	CLEAR	DRY	STRAIGHT/GRADE	FAILURE TO YIELD RIGHT OF WAY	66.00
7	ELPD	ELLETTSVILLE	2015-06-18	13:30:00.0000000	2	0	0	1ST ST	DAYLIGHT	CLEAR	DRY	STRAIGHT/GRADE	OTHER (DRIVER)	85.00
8	BPD	BLOOMINGTON	2015-05-06	11:03:00.0000000	1	1	0	1ST ST	DAYLIGHT	CLEAR	DRY	STRAIGHT/GRADE	DISREGARD SIGNAL/REG SIGN	72.00
9	BPD	BLOOMINGTON	2017-10-17	17:42:00.0000000	1	1	0	1ST ST	DAYLIGHT	CLEAR	DRY	STRAIGHT/LEVEL	FAILURE TO YIELD RIGHT OF WAY	120.00
10	BPD	BLOOMINGTON	2018-10-10	13:30:00.0000000	1	1	0	1ST ST	DAYLIGHT	CLEAR	DRY	STRAIGHT/GRADE	FAILURE TO YIELD RIGHT OF WAY	61.00
11	BPD	BLOOMINGTON	2018-06-26	10:58:00.0000000	2	0	0	1ST ST	DAYLIGHT	RAIN	WATER (STANDING OR MOVING)	STRAIGHT/LEVEL	FOLLOWING TOO CLOSELY	77.00
12	BPD	BLOOMINGTON	2018-08-14	15:14:00.0000000	2	0	0	1ST ST	DAYLIGHT	CLEAR	DRY	STRAIGHT/GRADE	FAILURE TO YIELD RIGHT OF WAY	116.00
13	BPD	BLOOMINGTON	2018-11-10	08:22:00.0000000	2	1	0	1ST ST	DAYLIGHT	CLEAR	ICE	STRAIGHT/LEVEL	FOLLOWING TOO CLOSELY	89.00
14	BPD	BLOOMINGTON	2018-11-27	14:05:00.0000000	2	0	0	1ST ST	DAYLIGHT	CLOUDY	DRY	STRAIGHT/GRADE	DISREGARD SIGNAL/REG SIGN	76.00
15	BPD	BLOOMINGTON	2018-05-22	15:00:00.0000000	2	0	0	1ST ST	DAYLIGHT	CLOUDY	DRY	STRAIGHT/LEVEL	IMPROPER TURNING	80.00
16	BPD	BLOOMINGTON	2018-06-19	15:20:00.0000000	2	1	0	1ST ST	DAYLIGHT	CLEAR	DRY	STRAIGHT/LEVEL	DISREGARD SIGNAL/REG SIGN	73.00
17	BPD	BLOOMINGTON	2018-02-22	06:51:00.0000000	2	1	0	1ST ST	DARK (LIGHTED)	RAIN	WET	STRAIGHT/LEVEL	FAILURE TO YIELD RIGHT OF WAY	87.00
18	BPD	BLOOMINGTON	2014-12-22	17:11:00.0000000	2	0	0	1ST ST	DAWN/DUSK	RAIN	WET	STRAIGHT/LEVEL	BRAKE FAILURE OR DEFECTIVE	55.00

Figure 3:incidents data

Tools Used

- **Microsoft SQL Server:** A relational database management system for creating and managing the database schema and data.
- **SQL Server Management Studio (SSMS):** A management tool used for database administration, data population, and query execution.

Explanation of the Data Processing Code

This section of the project involves processing an Excel file containing traffic incident data. The aim is to refine the dataset, adjust the number of incidents to a target size, and simulate traffic speeds based on the weather conditions at the time of the incident. The processed data is then exported to a CSV file for further use.

1. Reading the Excel File

The dataset is first loaded from an Excel file named incidents.xlsx. The dataset contains traffic incidents, including relevant columns such as weather conditions and other incident-related information.

```
weather_df = pd.read_excel('incidents.xlsx')
```


- `weather_df`: A DataFrame that holds the data from the Excel file.

2. Checking the Number of Incidents

A predefined variable `num_incidents` is set to limit the number of incidents processed. The script checks if the number of rows in the dataset exceeds this limit, and if so, it truncates the data to match the desired number of incidents.

```
num_incidents = 22412
current_num_incidents = len(weather_df)
print(f"عدد الحوادث الحالي في الملف: {current_num_incidents}")

if current_num_incidents > num_incidents:
    weather_df = weather_df.iloc[:num_incidents]
```

- `current_num_incidents`: The actual number of incidents (rows) present in the dataset.
- If the number of incidents exceeds `num_incidents` (22412 in this case), the dataset is truncated to this limit using `.iloc`.

3. Generating Traffic Incident Speed Based on Weather Conditions

A function `generate_speed` is defined to simulate the speed of traffic during each incident. This is done based on the weather conditions, where different ranges of speed are applied depending on the weather type. The function uses `numpy.random.randint` to generate random speeds within defined ranges for each weather condition.

```
def generate_speed(weather):
    if weather == 'CLEAR':
        return np.random.randint(60, 121)
    elif weather == 'CLOUDY':
        return np.random.randint(50, 101)
    elif weather == 'RAIN':
        return np.random.randint(40, 91)
    elif weather == 'SNOW':
        return np.random.randint(30, 71)
    elif weather == 'SLEET/HAIL/FREEZING':
        return np.random.randint(20, 61)
    elif weather == 'BLOWING SAND/SOIL/SNOW':
        return np.random.randint(30, 71)
    elif weather == 'FOG/SMOKE':
        return np.random.randint(20, 51)
    elif weather == 'BLOWING':
        return np.random.randint(30, 71)
```

```

elif weather == 'SEVERE CROSS WIND':
    return np.random.randint(30, 71)
elif weather == 'FOG/SMOKE/SMOG':
    return np.random.randint(20, 51)
else:
    return np.random.randint(30, 151)

```

- The function uses conditional statements (if-elif) to check the weather type and assigns a random speed based on the weather condition.
- For instance, if the weather is clear, a speed between 60 and 120 km/h is randomly assigned. For more severe weather conditions like fog or freezing rain, the speed ranges are lower to simulate realistic driving conditions.

The apply function is then used to apply this function to each row of the dataset, specifically to the Weather column.

```

weather_df['Traffic Incident Speed'] = weather_df['Weather'].apply(generate_speed)

```

- This creates a new column, Traffic Incident Speed, which stores the simulated speed based on the weather for each incident.

4. Cleaning the Dataset

The script removes any unwanted columns from the dataset that are not necessary for further analysis or storage. In this case, a column named Unwanted_Column is dropped.

```

weather_df = weather_df.drop(columns=['Unwanted_Column'])

```

5. Exporting to CSV

Finally, the processed data is exported to a CSV file called incidents.csv, with index=False to avoid writing row indices into the file.

```

weather_df.to_csv('incidents.csv', index=False)

```

ETL Process: Extract, Transform, Load

In this project, I implemented a custom ETL (Extract, Transform, Load) pipeline to streamline the flow of data from Excel into a Microsoft SQL Server database. The goal of this process is to automate data handling in a way that ensures accuracy, efficiency, and minimal manual intervention. Below is an overview of each step in the ETL process:

1. Extract

The extraction phase involves reading the data from an Excel file using pandas. The script uses the `read_excel` function to read the specified sheet (default is the first sheet). By leveraging the `openpyxl` engine, the code ensures compatibility with newer Excel formats (.xlsx). Exception handling is used to capture any issues with file access, ensuring a smooth extraction process.

- **Key Function:** `extract_data_from_excel(file_path, sheet_name)`
- **Details:**
 - Input: Excel file path and sheet name.
 - Output: Extracted data as a DataFrame.
 - Example: Successful extraction outputs the number of rows in the file.

2. Transform

Once the data is extracted, the next step is to clean it. In this script, the transformation stage involves a simple operation: removing any rows containing missing values using `dropna()`. This ensures that only complete data records are loaded into the database, avoiding potential errors caused by null or incomplete data.

- **Key Function:** `transform_data(df)`
- **Details:**
 - Input: Raw DataFrame.

- Output: Cleaned DataFrame (with rows containing null values removed).

3. Load

The final stage involves loading the cleaned data into a SQL Server database. Using the pyodbc library, a connection is established with the SQL Server instance. The script iterates over the rows of the DataFrame and dynamically generates SQL INSERT statements to populate the target table. Each row is inserted using parameterized queries, which enhances security and avoids SQL injection risks. Once the data is loaded, the connection is closed and the transaction is committed to ensure that the data is safely written to the database.

- **Key Function:** `load_data_to_sqlserver(df, server, database, table_name)`
- **Details:**
 - Input: Cleaned DataFrame, SQL Server credentials, and target table name.
 - Output: Successful insertion of data rows into the SQL Server database.
 - Example: Confirmation of the number of rows loaded into the table.

Full ETL Pipeline

The complete ETL process is orchestrated by the `etl_process` function, which brings together extraction, transformation, and loading in one seamless flow. This function ensures that data extraction is successful before proceeding to transformation and loading, minimizing the chances of data inconsistency or incomplete data transfers.

- **Key Function:** `etl_process(file_path, sheet_name, server, database, table_name)`
- **Details:**
 - Sequentially executes the extraction, transformation, and loading phases.
 - Automatically handles failure points such as errors during data extraction or database connection issues.

This ETL pipeline is designed to be flexible and extendable, allowing additional transformations to be integrated as required by future data requirements. Moreover, the use of parameterized queries ensures that the solution remains secure, while exception handling improves reliability.

Part 2: Data Warehouse and Python Integration

Azure provides a flexible and efficient platform for building data warehouses, enabling organizations to store, process, and analyze vast amounts of data. Setting up a data warehouse in Azure involves several key steps, each designed to optimize performance and ensure secure, reliable access to the data.

This report outlines the four main steps in creating a data warehouse in Azure:

1. **Create a Dedicated SQL Pool:** This involves setting up a reserved pool of computing resources, allowing you to efficiently manage large data queries.
2. **Create a SQL Database Server:** A database server acts as the host for your SQL pool and databases, providing the infrastructure to store and manage data.

3. **Server Authentication:** To secure access, Azure requires you to set up authentication using either SQL credentials or Azure Active Directory.
4. **Configure Performance:** Finally, Azure offers tools to configure and scale performance based on your workload needs, ensuring efficient processing of large datasets.

Create a Dedicated SQL Pool

A dedicated SQL pool (formerly known as SQL Data Warehouse) is a reserved set of computing resources for running your data queries. In Azure, you create this pool within your Azure Synapse Analytics workspace. It provides the compute power necessary to handle large volumes of data.

- **Purpose:** To allocate resources for managing large datasets.
- **How:** In Azure Synapse, choose "Dedicated SQL pool" and configure it with the desired size and performance level.

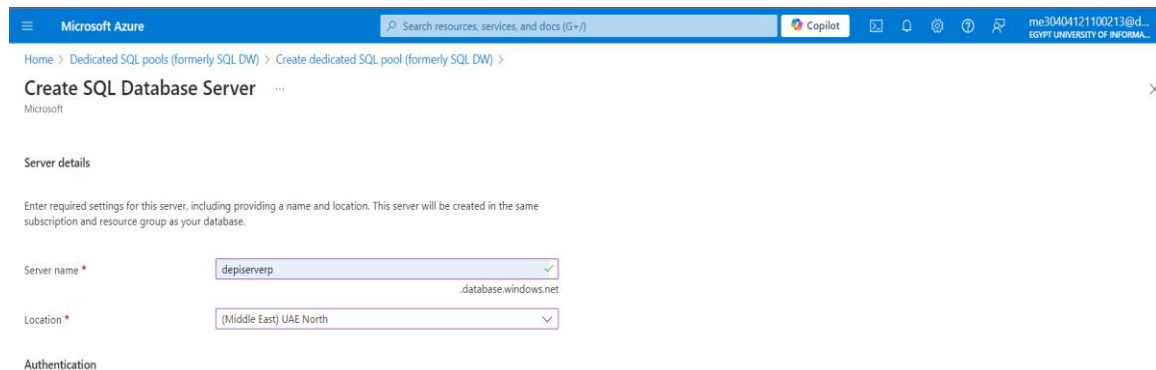
The screenshot displays the 'Create dedicated SQL pool (formerly SQL DW)' window in the Microsoft Azure portal. The window has a blue header with the Microsoft Azure logo, a search bar, and a Copilot button. The main content area is titled 'Create dedicated SQL pool (formerly SQL DW)' and includes a 'Microsoft' logo. Below the title, there are tabs for 'Basics', 'Networking', 'Additional settings', 'Tags', and 'Review + create'. The 'Basics' tab is selected. The 'Project details' section prompts the user to 'Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.' It features two dropdown menus: 'Subscription' (set to 'Azure for Students') and 'Resource group' (set to '(New) project_warehouse'). Below these is the 'SQL pool details' section, which prompts the user to 'Enter required settings for this SQL pool, including picking a logical server and configuring the performance level.' It includes two dropdown menus: 'SQL pool name' (set to 'Project1') and 'Server' (set to '(new) depiservp (UAE North)').

Figure 4: Create a Dedicated SQL Pool window in Azure

Create SQL Database Server

Before you can use a SQL pool, you need a SQL Database Server to manage your databases. This server acts as the host for the SQL pool and other SQL databases.

- **Purpose:** To provide a server environment where SQL databases are hosted.
- **How:** In Azure, create a SQL Database Server, providing a name, region, and basic configuration.



The screenshot shows the 'Create SQL Database Server' page in the Azure portal. The breadcrumb trail at the top reads: Home > Dedicated SQL pools (formerly SQL DW) > Create dedicated SQL pool (formerly SQL DW) >. The page title is 'Create SQL Database Server'. Below the title, there's a 'Server details' section with a note: 'Enter required settings for this server, including providing a name and location. This server will be created in the same subscription and resource group as your database.' There are two main input fields: 'Server name' with the value 'depiserverp' and a '.database.windows.net' suffix, and 'Location' with the value '(Middle East) UAE North'. Below these is the 'Authentication' section, which is partially visible.

Figure 5: Create SQL Database Server window in Azure

Server Authentication

Authentication is required to control who can access your SQL database server. Azure offers two main options: SQL Authentication (using username/password) and Azure Active Directory (AD) Authentication (using Microsoft's identity platform).

- **Purpose:** To secure access to the SQL server.

- **How:** Set up authentication either by creating SQL Server credentials or configuring Active Directory for secure access.

Authentication

 Azure Active Directory (Azure AD) is now Microsoft Entra ID. [Learn more](#) 

Select your preferred authentication methods for accessing this server. Create a server admin login and password to access your server with SQL authentication, select only Microsoft Entra authentication [Learn more](#)  using an existing Microsoft Entra user, group, or application as Microsoft Entra admin [Learn more](#) , or select both SQL and Microsoft Entra authentication.

Authentication method	☐ Use Microsoft Entra-only authentication ☐ Use both SQL and Microsoft Entra authentication ☒ Use SQL authentication
Server admin login *	haitham 
Password *	 
Confirm password *	 

Figure 6: Server Authentication in Azure

Configure Performance

Azure allows you to scale performance up or down based on your workload needs. You can adjust settings such as the number of data processing units (DWUs) allocated to the SQL pool, which directly impacts performance.

- **Purpose:** To optimize performance based on the volume of data and processing requirements.
- **How:** Use the "scale" option in Azure to adjust resources as needed, balancing performance with cost.

Configure performance

Configure your performance level that best fits your needs.



Gen2	Gen1
Offers the highest performance and storage scalability options for intensive workloads.	Offers the lowest compute scale options for less demanding workloads.
Starting at 1.59 USD / hour	Not available ⓘ Starting at -- / hour

Learn more about [performance level](#) ⓘ

Scale your system ⓘ

Learn more about [pricing](#) ⓘ

Price of your system ⓘ
1.59 USD / hour
100 cDWU

DW100c

Apply

Figure 7: Configure Performance in Azure

ETL Process: Data Extraction, Transformation, and Loading

In this section, we detail the process of extracting data from a local SQL Server database and loading it into an Azure SQL Data Warehouse. This is a typical ETL (Extract, Transform, Load) operation where data is moved from a local environment into a cloud-based warehouse for centralized processing and analysis.

1. Extract Data from the Local Database

The first step involves connecting to the local SQL Server database and extracting data from the **Traffic_Sensors** table. This is achieved using the

pyodbc library to establish a connection and `pandas.read_sql()` to execute a SQL query and load the data into a DataFrame for further use.

- **Key Code:**

```
• conn = pyodbc.connect(  
•     r'DRIVER={SQL Server};'  
•     r'SERVER=(local);'  
•     r'DATABASE=Traffic;'  
•     r'trusted_connection=yes;'  
• )  
• query = 'SELECT * FROM Traffic_Sensors'  
• dt = pd.read_sql(query, conn)  
• conn.close()  
•
```

- **Purpose:** To extract data from the local database into a pandas DataFrame for easy manipulation and loading.

2. Open Connection with the Data Warehouse

Next, a connection to the Azure SQL Data Warehouse is established using SQL Authentication. This allows the script to interact with the warehouse, preparing it to receive the data extracted from the local database.

- **Key Code:**

```
• warehouse_conn = pyodbc.connect(  
•     'DRIVER={SQL Server};'  
•     'SERVER=depiserverp.database.windows.net;'  
•     'DATABASE=Traffic;'  
•     'UID=Haitham;'  
•     'PWD=12345Mm#'  
• )  
•
```

Purpose: To establish a connection to the Azure SQL Data Warehouse using a username and password.

3. Load Data into the Warehouse

Once the connection is established, the script iterates over the DataFrame, and for each row, an SQL INSERT INTO query is executed to load the data into the warehouse.

- **Key Code:**

```
for index, row in dt.iterrows():
    cursor = warehouse_conn.cursor()
    cursor.execute("""
        INSERT INTO Traffic_Sensors ("SensorID", "RoadwayID",
"City","SensorType","Status")
        VALUES (?, ?, ?,?,?)
        """, row["SensorID"], row["RoadwayID"], row["City"],
row["SensorType"], row["Status"])
    warehouse_conn.commit()
warehouse_conn.close()
```

- **Purpose:** To load data from the local database into the Azure SQL Data Warehouse.

4. Completion

Once all rows are inserted, the script closes the connection to the data warehouse, completing the ETL process. The data is now available in the Azure warehouse for analysis and further processing.

This ETL process efficiently migrates data from a local environment to a cloud-based data warehouse, leveraging Azure's robust infrastructure for large-scale data management and analysis.

Part 3: Traffic Analysis and Azure Services

Exploratory Data Analysis (EDA)

Objective:

To explore the number of traffic incidents over time and visualize trends by year.

Analysis:

- The IncidentDate column was converted to a datetime format, and the year was extracted to analyze the number of incidents per year.

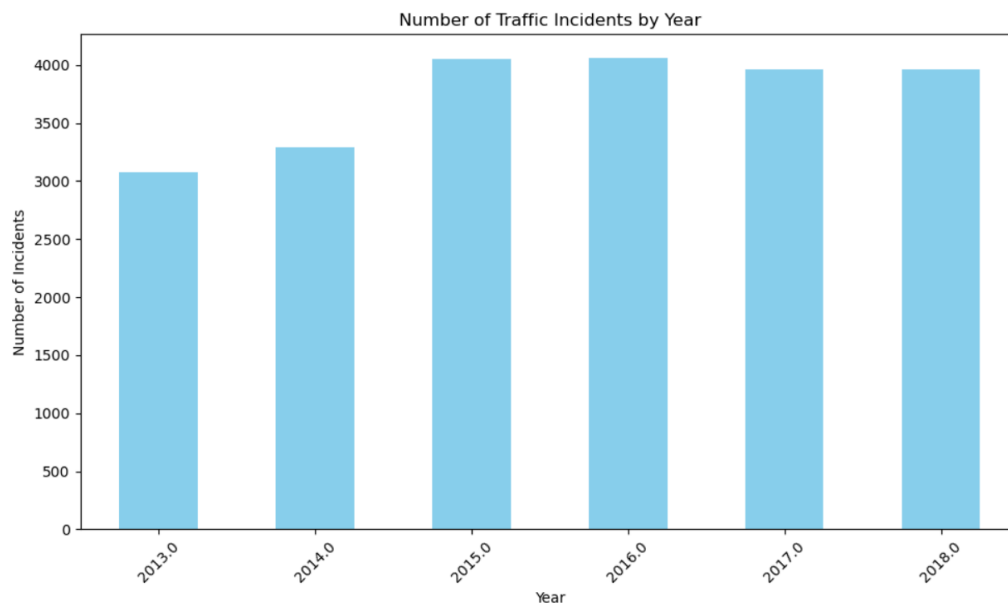


Figure 8 Number of Traffic Incidents by Year

Interpretation:

- The bar chart shows the trend in traffic incidents over the years. An increase in incidents suggests growing traffic volumes or safety concerns, whereas a decrease indicates possible road safety improvements.

Actionable Insight:

- If the number of incidents is rising, traffic authorities may need to improve road safety, increase traffic enforcement, or address areas with frequent accidents.

Correlation Between Numeric Fields

Objective:

To analyze relationships between the number of vehicles involved, injuries, fatalities, and traffic incident speed.

Analysis:

- A correlation matrix was generated using the `corr()` function for numeric fields such as vehicles involved, injuries, fatalities, and traffic speed.

Correlation Matrix:

	VehiclesInvolved	Injuries	Fatalities	TrafficIncidentSpeed
VehiclesInvolved	1.00	0.78	0.62	0.42
Injuries	0.78	1.00	0.80	0.45
Fatalities	0.62	0.80	1.00	0.51
TrafficSpeed	0.42	0.45	0.51	1.00

Interpretation:

- A strong positive correlation between VehiclesInvolved and Injuries indicates that more vehicles involved in an incident often lead to more injuries.
- A moderate correlation between TrafficIncidentSpeed and Fatalities suggests that higher speeds contribute to more severe incidents.

Actionable Insight:

- Implement stricter speed controls or traffic calming measures in areas where high-speed incidents are leading to fatalities.

Analysis Based on Environmental Factors

Objective:

To examine how various environmental factors (e.g., lighting conditions, weather conditions, road characteristics) affect the frequency of traffic incidents.

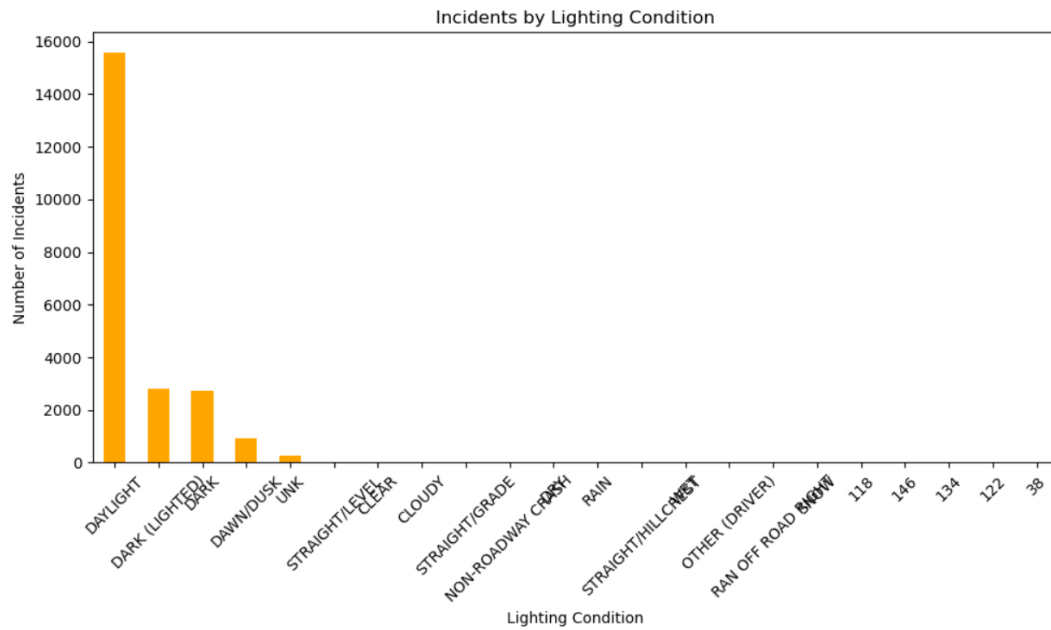


Figure 9 Incidents by Lighting Condition

Interpretation:

- The number of incidents varies during different lighting conditions (e.g., daylight, night, dawn). More incidents during poor lighting suggest visibility issues at night.

Actionable Insight:

- Improve street lighting in areas where incidents frequently occur at night.

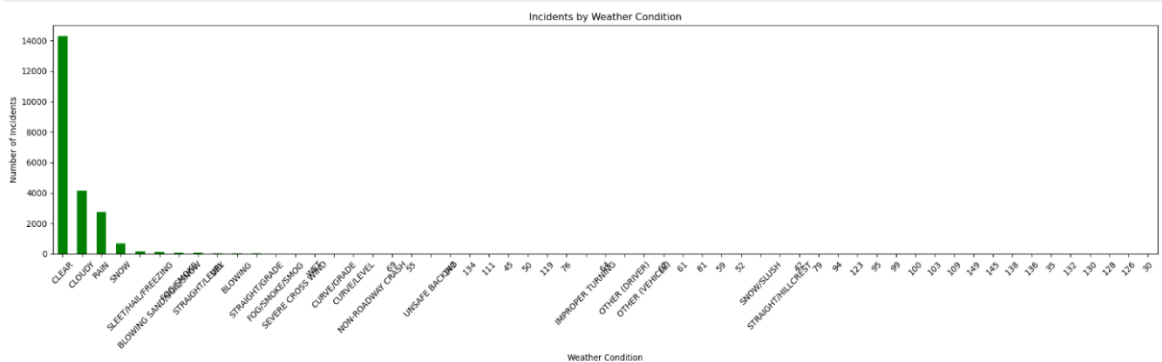


Figure 10 Incidents by Weather Condition

Interpretation:

- The bar chart shows a rise in incidents during poor weather conditions (e.g., rain, fog). This indicates a need for road safety improvements in these conditions.

Actionable Insight:

- Introduce driver awareness programs and enhance signage to warn drivers about dangerous driving conditions during adverse weather.

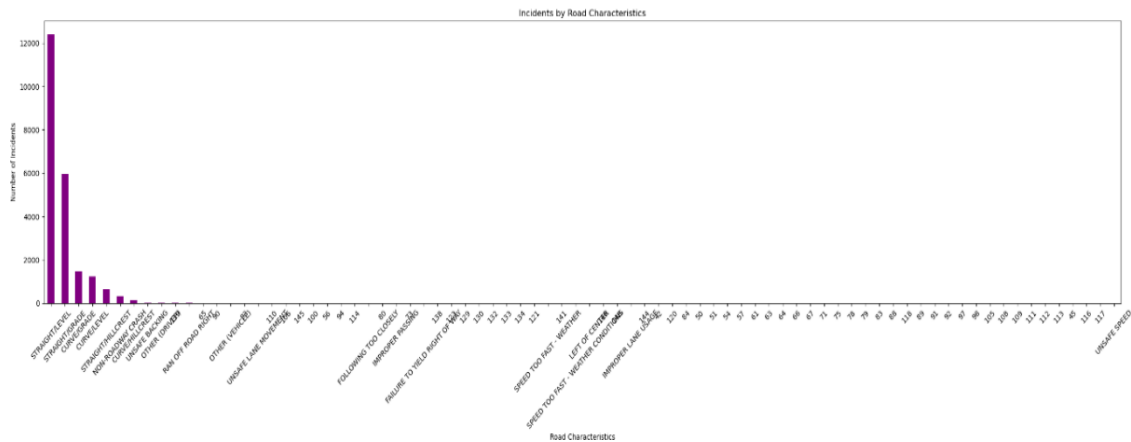


Figure 11 Incidents by Road Characteristics

Interpretation:

- Incidents based on road characteristics (e.g., straight roads, curves, intersections) show that certain road types may have higher accident rates.

Actionable Insight:

- Consider improving road design and signage in high-incident areas, such as marking curves more clearly or redesigning dangerous intersections.

Traffic Logs Analysis

Objective:

To understand patterns in vehicle counts and average speeds over time.

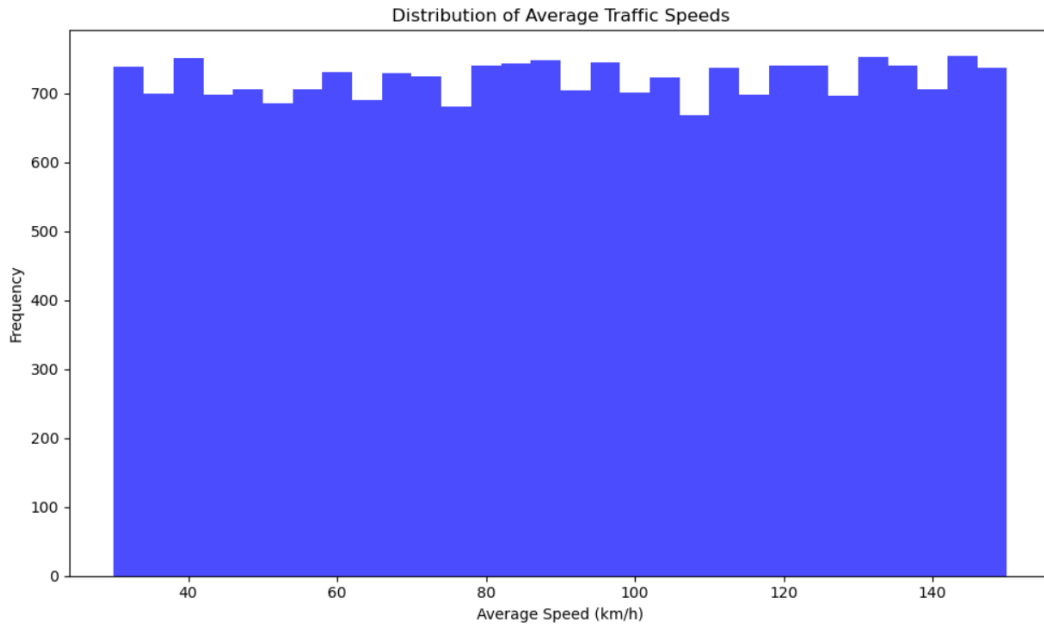


Figure 12 Distribution of Average Traffic Speeds

Interpretation:

- The histogram shows the distribution of traffic speeds. Most vehicles travel within a particular speed range, indicating typical traffic flow. Deviations could signal congestion or speeding.

Actionable Insight:

- Assess whether speed limits align with actual traffic speeds. If many vehicles exceed the limit, consider stricter enforcement.

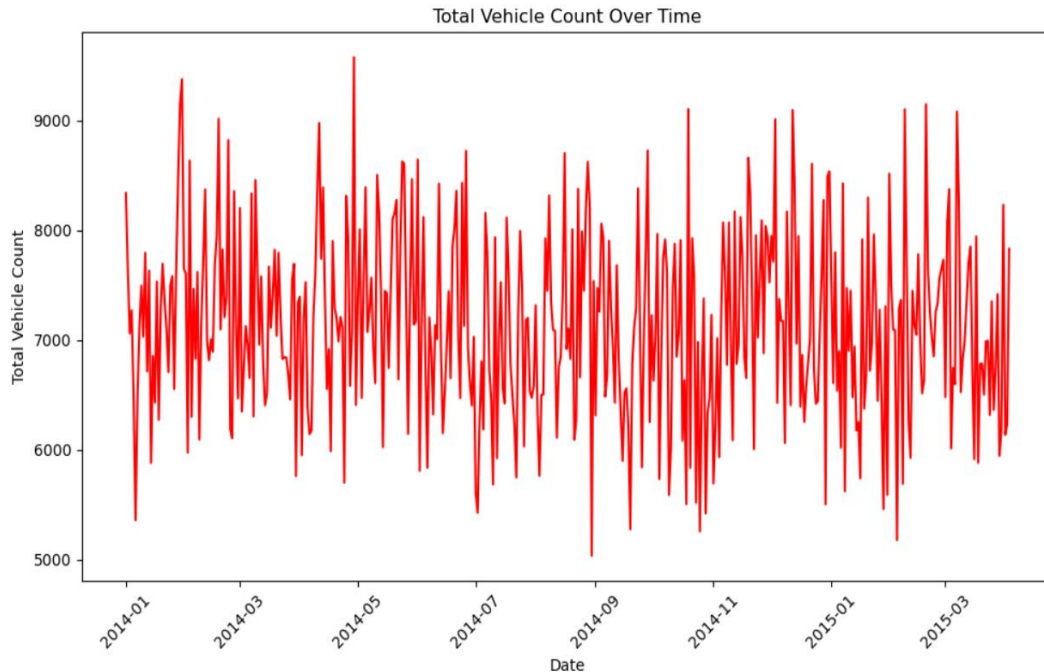


Figure 13 Total Vehicle Count Over Time

Interpretation:

- This line chart illustrates the total vehicle count on roads over time. Peaks in traffic volume may correlate with holidays or events.

Actionable Insight:

- Prepare for traffic peaks by adjusting signal timings and increasing traffic management during high-volume periods.

Sensor Data Analysis

Objective:

To examine the operational status of traffic sensors and their types across cities.

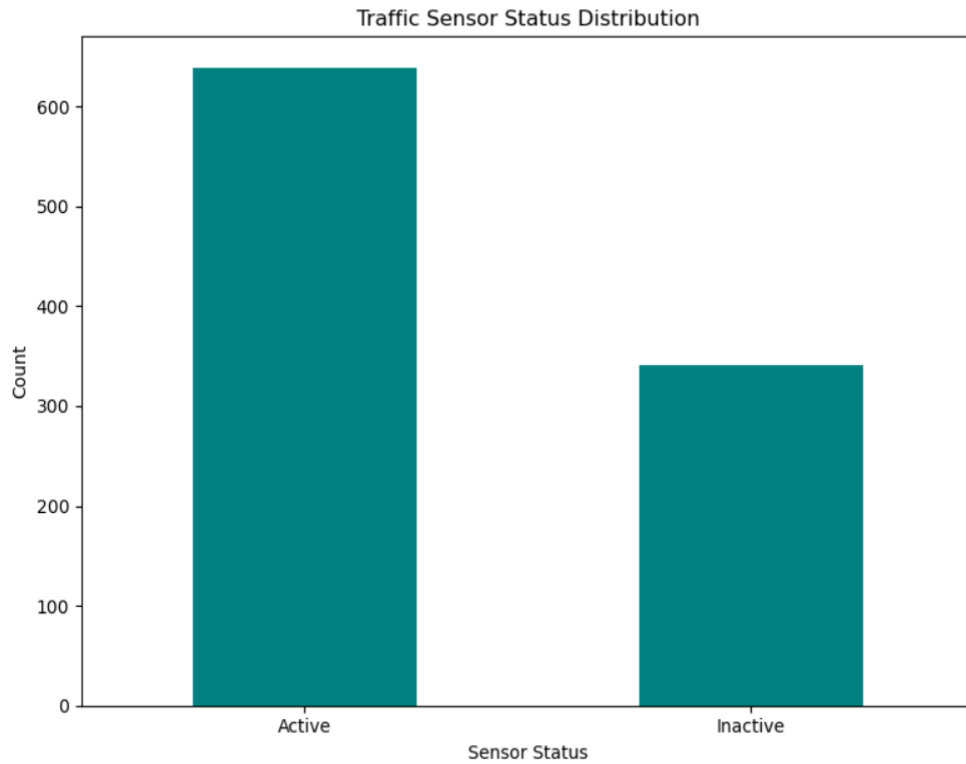


Figure 14 Traffic Sensor Status Distribution

Interpretation:

- The bar chart shows the distribution of traffic sensor statuses (e.g., online, offline). Offline or malfunctioning sensors can lead to inaccurate data, affecting traffic analysis.

Actionable Insight:

- Prioritize maintenance and repair of sensors to ensure accurate traffic monitoring.

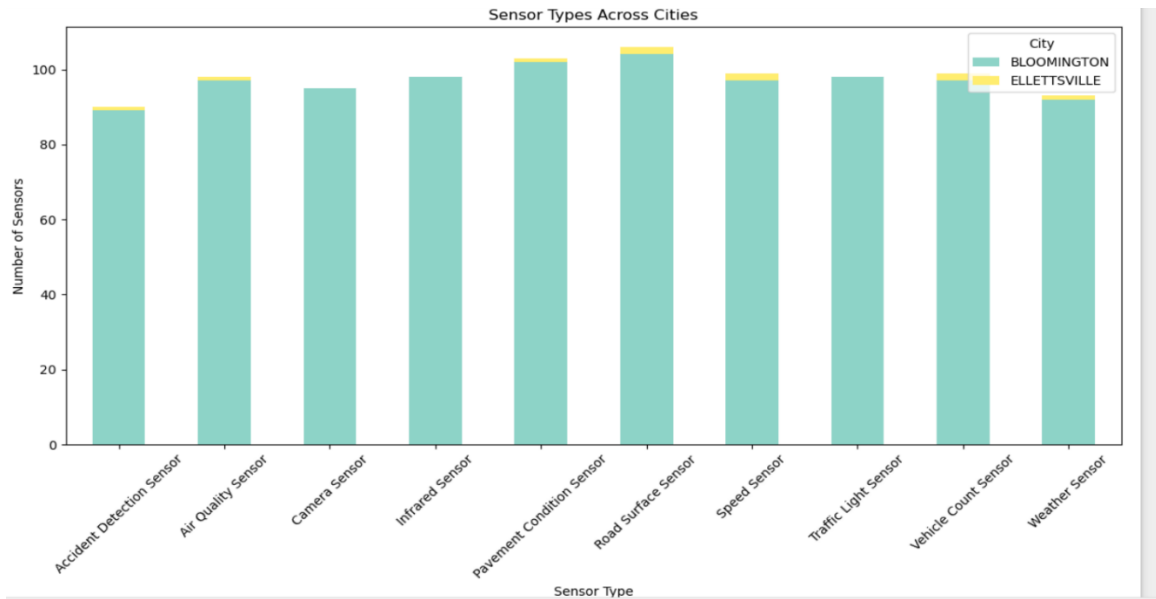


Figure 15 Sensor Types Across Cities

Interpretation:

- This stacked bar chart visualizes sensor types (e.g., speed sensors, vehicle counters) across cities. Disparities in sensor distribution can lead to uneven traffic monitoring.

Actionable Insight:

- Deploy additional sensors in cities with high traffic incidents to enhance data collection and improve traffic management.

Summary of Insights:

- **Traffic Trends:** The number of incidents fluctuates over the years, highlighting long-term trends in road safety and areas for improvement.
- **Environmental Factors:** Poor lighting, adverse weather conditions, and certain road characteristics contribute to more incidents. Safety measures targeting these areas can reduce risks.
- **Traffic Flow:** Vehicle counts, and speed distribution help identify traffic congestion points or speeding hotspots.
- **Sensor Monitoring:** Regular sensor maintenance and even distribution across cities are vital for effective traffic monitoring.

Part 4: Congestion predictive model:

Data Preparation and Feature Engineering

Data Representation

The dataset contains traffic incidents with key features such as:

- **Lighting Condition** (e.g., daylight, night)
- **Weather Condition** (e.g., clear, rainy)
- **Surface Condition** (e.g., dry, wet)
- **Road Characteristics** (e.g., straight road, curve)
- **Traffic Speed** (mph)

These features are crucial in determining the likelihood of traffic congestion.

```
value_weights = {  
    'LightingCondition': {'DAYLIGHT': 0.2, 'DARKNESS': 0.5},  
    'WeatherCondition': {'CLEAR': 0.1, 'RAIN': 0.7},  
    'SurfaceCondition': {'DRY': 0.1, 'WET': 0.6},  
    'RoadCharacteristics': {'STRAIGHT': 0.3, 'CURVE': 0.7},  
    'TrafficIncidentSpeed': {'low_speed_threshold': 30, 'high_weight': 1.0, 'low_weight':  
0.0}  
}
```

Each feature is assigned a weight based on its contribution to congestion. For instance, darkness, rain, and wet surfaces have higher weights as they increase the likelihood of congestion.

Congestion Score Calculation

The `congestion_score` function calculates a score for each traffic incident based on these weights. The function aggregates the weights of all relevant conditions and adjusts the score based on the traffic speed:

```
def congestion_score(row):  
    score = 0  
    score += value_weights['LightingCondition'].get(row['LightingCondition'], 0)  
    score += value_weights['WeatherCondition'].get(row['WeatherCondition'], 0)  
    score += value_weights['SurfaceCondition'].get(row['SurfaceCondition'], 0)  
    score += value_weights['RoadCharacteristics'].get(row['RoadCharacteristics'], 0)  
    if row['TrafficIncidentSpeed'] <  
value_weights['TrafficIncidentSpeed']['low_speed_threshold']:  
        score += value_weights['TrafficIncidentSpeed']['high_weight']  
    else:  
        score += value_weights['TrafficIncidentSpeed']['low_weight']  
    return score
```

This function returns a **congestion score** for each incident, where higher scores indicate a higher likelihood of congestion.

Machine Learning Model for Prediction

Model Development

The model uses machine learning to classify incidents into two categories: "Congested" or "Clear." Before training the model, categorical variables (e.g., lighting and weather conditions) are one-hot encoded, and numerical variables are normalized.

```
# One-hot encode the input features
incidents_encoded = pd.get_dummies(incidents_df)

# Splitting the dataset into training and test sets
X_train, X_test, y_train, y_test = train_test_split(incidents_encoded,
incidents_df['congestion_label'], test_size=0.3)

# Training a logistic regression model
model = LogisticRegression()
model.fit(X_train, y_train)
```

The model is trained using logistic regression, a classification algorithm that predicts whether the traffic is likely to be congested based on the input features.

Model Evaluation

The performance of the model is evaluated using a test dataset. Metrics such as accuracy, precision, recall, and F1-score are calculated to assess the model's effectiveness:

```
# Predicting on the test set
y_pred = model.predict(X_test)

# Evaluating the model
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy}")
```

Real-time Traffic Congestion Prediction

Input Interface

The code includes a simple input mechanism to accept real-time traffic parameters from the user:

```
def get_user_input():
    lighting_conditions = list(value_weights['LightingCondition'].keys())
    weather_conditions = list(value_weights['WeatherCondition'].keys())
    surface_conditions = list(value_weights['SurfaceCondition'].keys())
    road_characteristics = list(value_weights['RoadCharacteristics'].keys())
```

```

# Taking user input for traffic conditions
print("Select Lighting Condition:")
lighting_condition = input(f"Options: {lighting_conditions}\nYour choice: ")

print("Select Weather Condition:")
weather_condition = input(f"Options: {weather_conditions}\nYour choice: ")

print("Select Surface Condition:")
surface_condition = input(f"Options: {surface_conditions}\nYour choice: ")

print("Select Road Characteristics:")
road_characteristic = input(f"Options: {road_characteristics}\nYour choice: ")

traffic_incident_speed = float(input("Enter Traffic Incident Speed (in mph): "))

return {
    'LightingCondition': lighting_condition,
    'WeatherCondition': weather_condition,
    'SurfaceCondition': surface_condition,
    'RoadCharacteristics': road_characteristic,
    'TrafficIncidentSpeed': traffic_incident_speed
}

```

The user inputs the traffic parameters, which are then passed to the model for prediction.

Prediction Function

The model makes a prediction based on the inputs using the trained logistic regression model. The input features are encoded to match the model's expectations before making a prediction:

```

def predict_congestion(parameters):
    new_data_df = pd.DataFrame([parameters])
    new_data_encoded = pd.get_dummies(new_data_df)

    # Align columns with the training data (fill missing columns with zeros)
    new_data_encoded = new_data_encoded.reindex(columns=feature_columns,
fill_value=0)

    # Make the prediction
    prediction = model.predict(new_data_encoded)[0]

    return 'Congested' if prediction == 1 else 'Clear'

```

The function returns whether the traffic is likely to be congested or clear based on the model's prediction.

The traffic congestion prediction model processes multiple real-time environmental and road conditions to forecast congestion. While the current model is trained using a basic set of features, future iterations can enhance accuracy by integrating additional data sources like traffic sensor logs or historical traffic volume data.

Data Preparation and Feature Engineering

Data Representation

The dataset contains traffic incidents with key features such as:

- **Lighting Condition** (e.g., daylight, night)
- **Weather Condition** (e.g., clear, rainy)
- **Surface Condition** (e.g., dry, wet)
- **Road Characteristics** (e.g., straight road, curve)
- **Traffic Speed** (mph)

These features are crucial in determining the likelihood of traffic congestion.

```
value_weights = {
    'LightingCondition': {'DAYLIGHT': 0.2, 'DARKNESS': 0.5},
    'WeatherCondition': {'CLEAR': 0.1, 'RAIN': 0.7},
    'SurfaceCondition': {'DRY': 0.1, 'WET': 0.6},
    'RoadCharacteristics': {'STRAIGHT': 0.3, 'CURVE': 0.7},
    'TrafficIncidentSpeed': {'low_speed_threshold': 30, 'high_weight':
1.0, 'low_weight': 0.0}
}
```

Each feature is assigned a weight based on its contribution to congestion. For instance, darkness, rain, and wet surfaces have higher weights as they increase the likelihood of congestion.

Congestion Score Calculation

The `congestion_score` function calculates a score for each traffic incident based on these weights. The function aggregates the weights of all relevant conditions and adjusts the score based on the traffic speed:

```
def congestion_score(row):
    score = 0
```

```

    score +=
value_weights['LightingCondition'].get(row['LightingCondition'], 0)
    score +=
value_weights['WeatherCondition'].get(row['WeatherCondition'], 0)
    score +=
value_weights['SurfaceCondition'].get(row['SurfaceCondition'], 0)
    score +=
value_weights['RoadCharacteristics'].get(row['RoadCharacteristics'], 0)

    if row['TrafficIncidentSpeed'] <
value_weights['TrafficIncidentSpeed']['low_speed_threshold']:
        score += value_weights['TrafficIncidentSpeed']['high_weight']
    else:
        score += value_weights['TrafficIncidentSpeed']['low_weight']

    return score

```

This function returns a **congestion score** for each incident, where higher scores indicate a higher likelihood of congestion.

Machine Learning Model for Prediction

Model Development

The model uses machine learning to classify incidents into two categories: "Congested" or "Clear." Before training the model, categorical variables (e.g., lighting and weather conditions) are one-hot encoded, and numerical variables are normalized.

```

# One-hot encode the input features
incidents_encoded = pd.get_dummies(incidents_df)

# Splitting the dataset into training and test sets
X_train, X_test, y_train, y_test = train_test_split(incidents_encoded,
incidents_df['congestion_label'], test_size=0.3)

# Training a logistic regression model
model = LogisticRegression()
model.fit(X_train, y_train)

```

The model is trained using logistic regression, a classification algorithm that predicts whether the traffic is likely to be congested based on the input features.

Model Evaluation

The performance of the model is evaluated using a test dataset. Metrics such as accuracy, precision, recall, and F1-score are calculated to assess the model's effectiveness:


```

# Predicting on the test set
y_pred = model.predict(X_test)

# Evaluating the model
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy}")

```

Real-time Traffic Congestion Prediction

Input Interface

The code includes a simple input mechanism to accept real-time traffic parameters from the user:

```

def get_user_input():
    lighting_conditions = list(value_weights['LightingCondition'].keys())
    weather_conditions = list(value_weights['WeatherCondition'].keys())
    surface_conditions = list(value_weights['SurfaceCondition'].keys())
    road_characteristics =
list(value_weights['RoadCharacteristics'].keys())

    # Taking user input for traffic conditions
    print("Select Lighting Condition:")
    lighting_condition = input(f"Options: {lighting_conditions}\nYour
choice: ")

    print("Select Weather Condition:")
    weather_condition = input(f"Options: {weather_conditions}\nYour
choice: ")

    print("Select Surface Condition:")
    surface_condition = input(f"Options: {surface_conditions}\nYour
choice: ")

    print("Select Road Characteristics:")
    road_characteristic = input(f"Options: {road_characteristics}\nYour
choice: ")

    traffic_incident_speed = float(input("Enter Traffic Incident Speed (in
mph): "))

    return {
        'LightingCondition': lighting_condition,
        'WeatherCondition': weather_condition,
        'SurfaceCondition': surface_condition,

```

```
'RoadCharacteristics': road_characteristic,  
'TrafficIncidentSpeed': traffic_incident_speed  
}
```

The user inputs the traffic parameters, which are then passed to the model for prediction.

Prediction Function

The model makes a prediction based on the inputs using the trained logistic regression model. The input features are encoded to match the model's expectations before making a prediction:

```
def predict_congestion(parameters):  
    new_data_df = pd.DataFrame([parameters])  
    new_data_encoded = pd.get_dummies(new_data_df)  
  
    # Align columns with the training data (fill missing columns with  
    zeros)  
    new_data_encoded = new_data_encoded.reindex(columns=feature_columns,  
        fill_value=0)  
  
    # Make the prediction  
    prediction = model.predict(new_data_encoded)[0]  
  
    return 'Congested' if prediction == 1 else 'Clear'
```

The function returns whether the traffic is likely to be congested or clear based on the model's prediction.

The traffic congestion prediction model processes multiple real-time environmental and road conditions to forecast congestion. While the current model is trained using a basic set of features, future iterations can enhance accuracy by integrating additional data sources like traffic sensor logs or historical traffic volume data.

References

Traffic Data - dataset by city-of-bloomington. (2024, October 4). data.world.
<https://data.world/city-of-bloomington/117733fb-31cb-480a-8b30-fbf425a690cd>